7.2 Compilation and build

Dir 2.1	All source files shall compile without any compilation errors
Category	Required
Applies to	C90, C99

Rationale

A conforming compiler is permitted to produce an object module despite the presence of compilation errors. However, execution of the resulting program may produce unexpected behaviour.

See Section 5.3.1 for further guidance on controlling and analysing compiler messages.

See also

Rule 1.1

7.3 Requirements traceability

Dir 3.1	All code shall be traceable to documented requirements

[DO-178C Section 6.4.4.3.d]

Category	Required
Applies to	C90, C99

Rationale

Functionality that is not needed to meet the project requirements gives rise to unnecessary paths. It is possible that the developers of the software are not aware of the wider implications that might arise from this additional functionality. For example, developers might add code that toggles the state of a processor output pin every time a particular point in the program is reached. This might be very useful during development for measuring timing or for triggering an emulator or logic analyser. However, even though the pin in question might appear to be unused because it is not mentioned in the software requirements specification, it might be connected to an actuator in the target controller, resulting in unwanted external effects.

The method by which code is traced back to documented requirements shall be determined by the project. One method of achieving traceability is to review the code against the corresponding design documents which have in turn been reviewed against the requirements.

Note: there should not be any conflict between this guideline and the provision of a protective coding strategy as the latter should be part of the requirements in a critical system.

Licensed to: FH Campus Wien PFE GmbH.
21 Mar 2016. Copy 1 of 1

7.4 Code design

Dir 4.1 Run-time failures shall be minimized

C90 [Undefined 15, 19, 26, 30, 31, 32, 94]
C99 [Undefined 15, 16, 33, 40, 43–45, 48, 49, 113]

Category Required

Applies to C90, C99

Rationale

The C language was designed to provide very limited built-in run-time checking. While this approach allows generation of compact and fast executable code, it places the burden of run-time checking on the programmer. In order to achieve the desired level of robustness, it is therefore important that programmers carefully consider adding dynamic checks wherever there is potential for run-time errors to occur.

It is sometimes possible to demonstrate that the values of operands preclude the possibility of a run-time error during evaluation of an expression. In such cases, a dynamic check is not required provided that the argument supporting its omission is documented. Any such documentation should include the assumptions on which the argument depends. This information can be used during subsequent modifications of the code to ensure that the argument remains valid.

The techniques that will be employed to minimize run-time failures should be planned and documented, for example in design standards, test plans, static analysis configuration files and code review checklists. The nature of these techniques may depend on the integrity requirements of the project. See Section 5.2 for further details.

Note: the presence of a run-time error indicates a violation of Rule 1.3.

The following notes give some guidance on areas where consideration needs to be given to the provision of dynamic checks.

- arithmetic errors This includes errors occurring in the evaluation of expressions, such as overflow, underflow, divide by zero or loss of significant bits through shifting. In considering integer overflow, note that unsigned integer calculations do not strictly overflow but wrap around producing defined, but possibly unexpected, values. Careful consideration should be given to the ranges of values and order of operation in arithmetic expressions, for example:

```
float32_t f1 = 1E38f;
float32_t f2 = 10.0f;
float32_t f3 = 0.1f;
float32_t f4 = ( f1 * f2 ) * f3;   /* (f1 * f2) will overflow        */
float32_t f5 = f1 * ( f2 * f3 );   /* no overflow because (f2 * f3)
                                    * is (approximately) 1          */

if ( ( f3 >= 0.0f ) && ( f3 <= 1.0f ) )
{
  /*
   * no overflow because f3 is known to be in range 0..1 so the
   * result of the multiplication will fit in type float32_t
   */
  f4 = f3 * 100.0f;
}
```

- pointer arithmetic Ensure that when an address is calculated dynamically the computed address is reasonable and points somewhere meaningful. In particular it should be ensured that if a pointer points within an array, then when the pointer has been incremented or otherwise altered it still points within the same array. See restrictions on pointer arithmetic — Rule 18.1, Rule 18.2 and Rule 18.3.

Licensed to: FH Campus Wien PFE GmbH.
21 Mar 2016. Copy 1 of 1

- array bound errors Ensure that array indices are within the bounds of the array size before using them to index the array — Rule 18.1.
- function parameters The validity of arguments should be checked prior to passing them to library functions — Dir 4.11.
- pointer dereferencing Unless a pointer is already known to be non-NULL, a run-time check should be made before dereferencing that pointer. Once a check has been made, it is relatively straightforward within a single function to reason about whether the pointer may have changed and whether another check is therefore required. It is much more difficult to reason across function boundaries, especially when calling functions defined in other source files or libraries.

```
/*
 * Given a pointer to a message, check the message header and return
 * a pointer to the body of the message or NULL if the message is
 * invalid.
 */
const char *msg_body ( const char *msg )
{
  const char *body = NULL;

  if ( msg != NULL )
  {
    if ( msg_header_valid ( msg ) )
    {
      body = &msg[ MSG_HEADER_SIZE ];
    }
  }
  return body;
}

      char  msg_buffer[ MAX_MSG_SIZE ];
const char *payload;

payload = msg_body ( msg_buffer );

/* Check if there is a payload */
if ( payload != NULL )
{
  /* Process the payload */
}
```

- dynamic memory If dynamic memory allocation is being performed, it is essential to check that each allocation succeeds and that an appropriate degradation or recovery strategy is designed and tested.

See also

Dir 4.11, Dir 4.12, Rule 1.3, Rule 18.1, Rule 18.2, Rule 18.3

Licensed to: FH Campus Wien PFE GmbH.
21 Mar 2016. Copy 1 of 1

Dir 4.6	*typedefs* that indicate size and signedness should be used in place of the basic numerical types
Category	Advisory
Applies to	C90, C99

Amplification

The basic numerical types of *char*, *short*, *int*, *long*, *long long* (C99), *float*, *double* and *long double* should not be used, but specific-length *typedef*s should be used.

For C99, the types provided by `<stdint.h>` should be used. For C90, equivalent types should be defined and used.

A type must not be defined with a specific length unless the implemented type is actually of that length.

It is not necessary to use *typedef*s in the declaration of bit-fields.

Licensed to: FH Campus Wien PFE GmbH.
21 Mar 2016. Copy 1 of 1

For example, on a 32-bit C90 implementation the following definitions might be suitable:

```
typedef signed   char   int8_t;
typedef signed   short  int16_t;
typedef signed   int    int32_t;
typedef signed   long   int64_t;

typedef unsigned char   uint8_t;
typedef unsigned short  uint16_t;
typedef unsigned int    uint32_t;
typedef unsigned long   uint64_t;

typedef          float  float32_t;
typedef          double float64_t;
typedef long     double float128_t;
```

Rationale

In situations where the amount of memory being allocated is important, using specific-length types makes it clear how much storage is being reserved for each object.

Adherence to this guideline does **not** guarantee portability because the size of the *int* type may determine whether or not an expression is subject to integer promotion. For example, an expression with type `int16_t` will not be promoted if *int* is implemented using 16 bits but will be promoted if *int* is implemented using 32 bits. This is discussed in more detail in the section on integer promotion in Appendix C.

Note: defining a specific-length type whose size is **not** the same as the implemented type is counter-productive both in terms of storage requirements and in terms of portability. Care should be taken to avoid defining types with the wrong size.

If abstract types are defined in terms of a specific-length type then it is not necessary, and may even be undesirable, for those abstract types to specify the size or sign. For example, the following code defines an abstract type representing mass in kilograms but does not indicate its size or sign:

```
typedef uint16_t mass_kg_t;
```

It might be desirable not to apply this guideline when interfacing with The Standard Library or code outside the project's control.

Exception

1. The basic numerical types may be used in a *typedef* to define a specific-length type.
2. For function `main`, an *int* may be used rather than the *typedef*s as a return type. Therefore `int main (void)` is permitted.
3. For function `main` an *int* may be used rather than the *typedef*s for the input parameter `argc`.
4. For function `main` a *char* may be used rather than the *typedef*s for the input parameter `argv`.

Therefore `int main( int argc, char *argv[] )` is permitted (C99 Section 5.1.2.2.1).

Example

```
/* Non-compliant - int used to define an object                    */
int x = 0;

/* Compliant     - int used to define specific-length type         */
typedef int SINT_16;
```


Licensed to: FH Campus Wien PFE GmbH.
21 Mar 2016. Copy 1 of 1

```
/* Non-compliant - no sign or size specified                          */
typedef int speed_t;

/* Compliant     - further abstraction does not need specific length */
typedef int16_t torque_t;
```

Dir 4.7 If a function returns error information, then that error information shall be tested

Category Required

Applies to C90, C99

Amplification

The list of functions that are deemed to return error information shall be determined by the project.

The error information returned by a function shall be tested in a meaningful manner.

Rationale

A function (whether it is part of The Standard Library, a third party library or a user defined function) may be deemed to provide some means of indicating the occurrence of an error. This may be via an error flag, some special return value or some other means. Whenever such a mechanism is provided by a function the calling program shall check for the indication of an error as soon as the function returns.

However, note that the checking of input values to functions is considered a more robust means of error prevention than trying to detect errors after the function has completed (see Dir 4.11).

Exception

If it can be shown, for example by checking arguments, that a function cannot return an error indication then there is no need to perform a check.

See also

Dir 4.11, Rule 17.7

Licensed to: FH Campus Wien PFE GmbH.
21 Mar 2016. Copy 1 of 1

Dir 4.10	Precautions shall be taken in order to prevent the contents of a *header file* being included more than once
Category	Required
Applies to	C90, C99

Rationale

When a translation unit contains a complex hierarchy of nested *header files*, it is possible for a particular *header file* to be included more than once. This can be, at best, a source of confusion. If this multiple inclusion leads to multiple or conflicting definitions, then this can result in undefined or erroneous behaviour.

Example

```
/* file.h */
#ifndef FILE_H
/* Non-compliant - does not #define FILE_H */
#endif
```

In order to facilitate checking, the contents of the header should be protected from being included more than once using one of the following two forms:

Licensed to: FH Campus Wien PFE GmbH.
21 Mar 2016. Copy 1 of 1

```
<start-of-file>
#if !defined ( identifier )
#define identifier
  /* Contents of file */
#endif
<end-of-file>

<start-of-file>
#ifndef identifier
#define identifier
  /* Contents of file */
#endif
<end-of-file>
```

Note: the identifier used to test and record whether a given *header file* has already been included shall be unique across all *header files* in the project.

Note: comments are permitted anywhere within these forms.

Dir 4.11 The validity of values passed to library functions shall be checked

C90 [Undefined 60, 63, 96; Implementation 45–47]
C99 [Unspecified 30, 31, 44, 48–50;
Undefined 102, 103, 107, 112, 180, 181, 183, 187, 189;
Implementation J.3(8–11)]

Category Required

Applies to C90, C99

Amplification

The nature and organization of the project will determine which libraries, and functions within those libraries, should be subject to this directive.

Rationale

Many functions in The Standard Library are not required by The Standard to check the validity of parameters passed to them. Even where checking is required by The Standard, or where compiler writers claim to check parameters, there is no guarantee that adequate checking will take place.

Similarly, the interface description for functions in other libraries may not specify the checks performed by those functions. There is also a risk that the specified checks are not performed adequately.

The programmer shall provide appropriate checks of input values for all library functions which have a restricted input domain (The Standard Library, third-party libraries, and in-house libraries).

Examples of functions from The Standard Library that have a restricted domain and need checking are:

- Many of the maths functions in `<math.h>`, for example:
  - Negative numbers must not be passed to the *sqrt* or *log* functions;
  - The second parameter of *fmod* should not be zero;
- Some implementations can produce unexpected results when the function *toupper* is passed an argument which is not a lowercase letter (and similarly for *tolower*);
- The character testing functions in `<ctype.h>` exhibit undefined behaviour if passed invalid values;
- The *abs* function applied to the most negative integer gives undefined behaviour.

Licensed to: FH Campus Wien PFE GmbH.
21 Mar 2016. Copy 1 of 1

Although most of the math library functions in <math.h> define allowed input domains, the values they return when a domain error occurs may vary from one compiler to another. Therefore pre-checking the validity of the input values is particularly important for these functions.

The programmer should identify any domain constraints which should sensibly apply to a function being used (which may or may not be documented in the interface description), and provide appropriate checks that the input value(s) lies within this domain. Of course the value may be restricted further, if required, by knowledge of what the parameter represents and what constitutes a sensible range of values for the parameter.

There are a number of ways in which the requirements of this guideline might be satisfied, including the following:

- Check the values before calling the function;
- Check the values in the called library function — this is particularly applicable for in-house designed libraries, though it could apply to bought-in libraries if the supplier can demonstrate that they have built in the checks;
- Produce "wrapped" versions of functions that perform the checks then call the original function;
- Demonstrate statically that the input parameters can never take invalid values.

See also

Dir 4.1, Dir 4.7

Licensed to: FH Campus Wien PFE GmbH.
21 Mar 2016. Copy 1 of 1

Rule 14.2 A *for* loop shall be well-formed

Category	Required
Analysis	Undecidable, System
Applies to	C90, C99

Amplification

The three clauses of a *for* statement are the:

First clause which

- Shall be empty, or
- Shall assign a value to the *loop counter*, or
- Shall define and initialize the *loop counter* (C99).

Second clause which

- Shall be an expression that has no *persistent side effects*, and
- Shall use the *loop counter* and optionally *loop control flags*, and
- Shall not use any other object that is modified in the *for* loop body.

Third clause which

- Shall be an expression whose only *persistent side effect* is to modify the value of the *loop counter*, and
- Shall not use objects that are modified in the *for* loop body.

There shall only be one *loop counter* in a *for* loop, which shall not be modified in the *for* loop body.

A *loop control flag* is defined as a single identifier denoting an object with *essentially Boolean type* that is used in the *Second clause*.

The behaviour of a *for* loop body includes the behaviour of any functions called within that statement.

Rationale

The *for* statement provides a general-purpose looping facility. Using a restricted form of loop makes code easier to review and to analyse.

Exception

All three clauses may be empty, for example `for ( ; ; )`, so as to allow for infinite loops.

Licensed to: FH Campus Wien PFE GmbH.
21 Mar 2016. Copy 1 of 1

Example

In the following C99 example, `i` is the *loop counter* and `flag` is a *loop control flag*.

```
bool_t flag = false;

for ( int16_t i = 0; ( i < 5 ) && !flag; i++ )
{
  if ( C )
  {
    flag = true;                /* Compliant - allows early termination
                                 * of loop                                */
  }

  i = i + 3;                    /* Non-compliant - altering the loop
                                 * counter                                */
}
```

See also

Rule 14.1, Rule 14.3, Rule 14.4

Licensed to: FH Campus Wien PFE GmbH.
21 Mar 2016. Copy 1 of 1

8.15 Control flow

Rule 15.1	The *goto* statement should not be used
Category	Advisory
Analysis	Decidable, Single Translation Unit
Applies to	C90, C99

Rationale

Unconstrained use of *goto* can lead to programs that are unstructured and extremely difficult to understand.

In some cases a total ban on *goto* requires the introduction of flags to ensure correct control flow, and it is possible that these flags may themselves be less transparent than the *goto* they replace. Therefore, if this rule is not followed, the restricted use of *goto* is allowed where that use follows the guidance in Rule 15.2 and Rule 15.3.

See also

Rule 9.1, Rule 15.2, Rule 15.3, Rule 15.4

Licensed to: FH Campus Wien PFE GmbH.
21 Mar 2016. Copy 1 of 1